

Reverse Engineering Skill

User Guide

A GitHub Copilot Agent Skill that reverse engineers any GitHub repository or local project into a complete architecture, authentication, business-logic, and screen-navigation report — no Anthropic API key required.

Version 3.2.0
skill/reverse-engineering-skill/

Contents

1. Overview
2. Requirements
3. Installation
4. Quick Usage
5. Command-Line Reference
6. Output Files Reference
7. Authentication Detection Reference (RBAC / ABAC / ReBAC)
8. How the Pipeline Works
9. Repository Layout
10. Troubleshooting

1. Overview

The Reverse Engineering Skill is a GitHub Copilot Agent Skill that analyzes any GitHub repository, or any project already sitting on your local disk, and produces a focused 4-section report. GitHub Copilot Chat itself is the AI engine — the bundled Python analyzer needs nothing beyond the standard library, so no Anthropic API key or external LLM account is required.

The four report sections

#	Section	Covers
1	System Design Overview	Architecture, codebase metrics, API catalog, data model, dependency graph, modernization roadmap
2	Authentication & Access Control	RBAC, ABAC, ReBAC detection with named roles, policies, and a route protection map
3	Business Logic Extractor	Business domain, end-to-end workflows, user roles, business rules, entity glossary
4	Screen-by-Screen Navigation	Complete UI screen inventory and Mermaid navigation flowchart (ASPX, Razor, React, Vue, JSP)

Design principles

- **Remote or local, same pipeline** — a GitHub URL is shallow-cloned to a temp directory and deleted after analysis; a local folder is read in place and never modified, moved, or deleted.
- **Self-excluding** — when analyzing a local project, the skill's own installed folder (`.github/skills/*`) is always pruned from the walk.
- **One artifact to read** — the dependency graph and navigation flow render as Mermaid diagrams inline inside the Markdown report, so it displays natively on GitHub, GitLab, and the VS Code preview pane with no extra files to open.

2. Requirements

Tool	Version
VS Code	1.90 or later
GitHub Copilot Chat extension	latest
Python	3.8 or later
Git	any version, on PATH — only needed when analyzing a GitHub URL; not required for local-folder analysis

3. Installation

Step 1 — Copy the skill folder into your project

```
Your project/
|-- .github/
    |-- skills/
        |-- reverse-engineering-skill/  <- paste this folder here
            |-- SKILL.md
            |-- manifest.yaml
            |-- scripts/
            |-- assets/
            |-- references/
            |-- templates/
```

If `.github/skills/` doesn't exist yet, create it.

Step 2 — Verify Python is installed

```
python --version
# Should print Python 3.8 or higher
```

Step 3 — Done. Open GitHub Copilot Chat and use the skill.

4. Quick Usage

Analyze a GitHub repository

Reverse engineer <https://github.com/owner/repo>

Analyze a project already on your local disk

No cloning, no internet access — the folder is read in place and never modified.

```
Reverse engineer this project
Analyze the codebase in C:\Projects\LegacyApp
Reverse engineer ./my-app
```

Copilot will then:

- Ask where to save the output files
- Run the Python analysis engine automatically (clones if it's a URL, reads in place if it's a local folder)
- Produce AI-quality narrative for all 4 sections
- Write the final report directly to your chosen location

Choosing where output goes

Option	Result
Press Enter / say nothing	Default — a folder named after the repo/project in the current directory, e.g. <code>./nopCommerce/</code>
"save in current folder"	Files land directly next to your project files (<code>--output .</code>)

"save to C:\Reports"	Writes to the custom path you name
----------------------	------------------------------------

5. Command-Line Reference

The engine can also be run directly, without Copilot. It auto-detects whether the target is a remote URL (`https://`, `http://`, `git://`, `user@host:...`) or an existing local directory — no separate flag needed.

```
python .github/skills/reverse-engineering-skill/scripts/reverse_engineer_skill.py \  
    <TARGET> [--heuristic | --ai] [--output <PATH>]
```

Argument	Meaning
<TARGET>	A GitHub URL, or a path to an existing local directory
--heuristic	Pure static heuristics — no API calls, no API key (default, recommended)
--ai	Uses the Claude API for richer analysis — requires ANTHROPIC_API_KEY and the anthropic Python SDK
--output <PATH>	Where to write output files. Omit for the default <code>./{repo_name}/</code> ; pass <code>.</code> for the current directory; pass any path for a custom location

Examples

```
# Remote – default output → ./{repo-name}/ in current directory  
python .github/skills/reverse-engineering-skill/scripts/reverse_engineer_skill.py \  
    https://github.com/owner/repo --heuristic  
  
# Remote – output directly to current directory  
python .github/skills/reverse-engineering-skill/scripts/reverse_engineer_skill.py \  
    https://github.com/owner/repo --heuristic --output .  
  
# Local folder – analyzed in place, nothing cloned or deleted  
python .github/skills/reverse-engineering-skill/scripts/reverse_engineer_skill.py \  
    C:\Projects\LegacyApp --heuristic  
  
# Local folder – the current directory itself, output to a sibling folder  
python .github/skills/reverse-engineering-skill/scripts/reverse_engineer_skill.py \  
    . --heuristic --output ../legacyapp-analysis
```

6. Output Files Reference

File	Contents	Primary consumer
{repo}_report.md	4-section focused report — System Design (incl. ASCII block diagram + Mermaid dependency graph), Auth, Business Logic, Screens	Human stakeholders, Copilot AI narrative
{repo}_sdd.json	Full JSON System Design Document — 16 sections including auth_analysis and screens_navigation	AI assistants, tooling

{repo}_evaluation.md	100-point quality score with a confidence band (HIGH / MEDIUM / LOW)	QA, architects
manifest.json	Run metrics: files, classes, methods, endpoints, timestamps	CI, tooling

No SVG diagrams or HTML dashboard are generated — the dependency graph and navigation flow render as Mermaid diagrams directly inside the Markdown report, so it displays natively on GitHub, GitLab, and the VS Code preview pane with no extra files to open. See a real worked example in [marketplace/SampleOutputs/](#).

7. Authentication Detection Reference

The engine scans class names, method names, imports, and route decorators for three authorization models, and reports whichever combination it finds as a hybrid.

RBAC — Role-Based Access Control

“What role do you have?” → determines what you can do.

Framework	Pattern
ASP.NET Core	<code>[Authorize(Roles="Admin,Manager")] · User.IsInRole("Admin")</code>
Spring (Java)	<code>@PreAuthorize("hasRole('ADMIN')") · @Secured({"ROLE_ADMIN"})</code>
Flask (Python)	<code>@login_required · @role_required("admin")</code>
Django (Python)	<code>@permission_required("myapp.can_publish")</code>
Express/NestJS (TS)	<code>@UseGuards(RolesGuard) · @Roles("admin")</code>

ABAC — Attribute/Policy-Based Access Control

“What attributes do you and the resource have?” → determines access.

Framework	Pattern
ASP.NET Core	<code>[Authorize(Policy="CanEditOrder")] · services.AddAuthorization(...)</code>
Spring (Java)	<code>@PreAuthorize("hasPermission(...)") · @PostAuthorize</code>
Django (Python)	<code>ObjectPermission · has_object_permission(...)</code>

ReBAC — Relationship-Based Access Control

“What is your relationship to this resource?” → determines access.

Framework	Pattern
Any	<code>IsOwner(resource, user) · CreatedBy == User.Id</code>
ASP.NET Core	<code>order.UserId != User.GetUserId() · AuthorizationHandler<TReq, TResource></code>
Spring (Java)	<code>@PreAuthorize("hasPermission(#order, 'EDIT')")</code>
Django (guardian)	<code>@permission_required_or_403("orders.change_order", ...)</code>

Hybrid models

Combination	Description
RBAC + ABAC	Role gates coarse access; policy fine-tunes it
RBAC + ReBAC	Role gates entry; ownership check gates edit (e.g. admin sees all, regular user only their own)

All three	Enterprise pattern — Admin (RBAC) → Policy check (ABAC) → Ownership (ReBAC)
-----------	---

Security observations the report flags

- Inconsistent enforcement — some controllers guarded, others not
- Sensitive routes without guards — /admin/*, /api/users, /payments/* left public
- Over-permissive roles — a single “Admin” role with no granularity
- Missing ABAC — business rules enforced ad-hoc in code instead of via policies
- No ReBAC — users can potentially access other users’ resources if IDs are guessable

8. How the Pipeline Works

```
reverse_engineer_skill.py (CLI entry point)
↓
engine/pipeline.py run_pipeline(repo_url, mode, output_dir)
↓
[1] Clone (remote) or use in place (local)
[2] load_repo()                – loaders.py – walks the source tree
[3] parse_file()               – parsers.py – per-language extraction
[4] generate_report()           – analyzer.py – codebase metrics
[5] extract_api_endpoints()     – analyzer.py – route detection
[6] detect_auth_patterns()     – analyzer.py – RBAC / ABAC / ReBAC
[7] detect_screens_nav()       – analyzer.py – UI screen inventory
[8] ai_*() heuristics           – ai_analysis.py – domain analysis, no API calls
[9] generate_sdd() / generate_md_report() – generators/
[10] OutputManager              – output_manager.py – writes files
[11] evaluate_pipeline_output   – evaluator.py – 100-point quality score
```

Supported languages

Extension	Language	Parser function
.py	Python	parse_python()
.java	Java	parse_java()
.cs, .aspx.cs	C# / .NET	parse_dotnet()
.ts, .tsx	TypeScript	parse_js_ts()
.js, .jsx	JavaScript	parse_js_ts()
.vb	VB.NET	parse_dotnet()

File cap for large repositories

For repos over 300 files, the engine samples intelligently by architectural layer so the most informative files are always included:

Layer	Slots	Rationale
Controllers / Handlers	75	Entry points — API surface
Services / Managers	75	Business logic
Repositories / DAOs	40	Data access
Domain / Entities	60	Database schema
Models / DTOs	30	Data shapes
Everything else	20	Infrastructure

9. Repository Layout

This repository follows the team's standard marketplace skill layout:

```
reverse-engineering-skill-github-copilot/
|-- skill/
|   |-- reverse-engineering-skill/  <- copy this into .github/skills/
|       |-- SKILL.md
|       |-- manifest.yaml
|       |-- scripts/
|       |-- assets/
|       |-- references/
|       |-- templates/
|-- marketplace/                  <- this folder - docs, guide, samples
|   |-- README.md
|   |-- QuickStart.md
|   |-- UserGuide.pdf             (this document)
|   |-- Architecture.png
|   |-- SamplePrompts.md
|   |-- SampleOutputs/
```

10. Troubleshooting

Symptom	Likely cause / fix
"python: command not found"	Python 3.8+ isn't installed or isn't on PATH. Install from python.org and re-open your terminal.
Script not found under .github/skills/...	The skill folder wasn't copied in, or was copied to the wrong path. Re-check Section 3 — Installation.
"neither a git URL nor an existing local directory"	The target string didn't match a recognized URL scheme and isn't a real path on disk. Check spelling and that the path exists.
git clone fails for a remote URL	Git isn't installed / not on PATH, the repo is private without credentials configured, or there's no network access. Local-folder analysis doesn't need Git at all.
Output folder nested inside the analyzed project	Expected when analyzing "this project" with the default output option — harmless, since generated .md/.json files aren't re-analyzed as source on the next run. Say "save to ../my-project-analysis" for a clean split.
Auth section says "None detected"	The codebase doesn't match any of the known RBAC/ABAC/ReBAC patterns in Section 7 — common for auth implemented via a fully custom or third-party middleware the pattern library doesn't yet cover.
--ai mode falls back to heuristic	ANTHROPIC_API_KEY isn't set, or the anthropic SDK isn't installed, or the API call failed. The pipeline automatically continues with --heuristic so a report is still produced.